Operating Systems: CSE 3204

Chapter Two
Process Management

Lecture 3: Threads

ASTU
Department of CSE

Threads

- Inter-process communication is simple and easy when used occasionally
  - If there are many processes sharing many resources, then the mechanism becomes cumbersome- difficult to handle.
    - ➢ Threads are created to make this kind of resource sharing simple & efficient
- A thread is a basic unit of CPU utilization that consists of:
    - thread id
    - program counter
    - register set
    - stack
- Threads belonging to the same process **share**:
    - its code
    - its data section
    - other OS resources

Processes and Threads

- A **thread of execution** is the smallest unit of processing that can be scheduled by an OS.
- The implementation of threads and process differs from one OS to another, but in most cases, **a thread is contained inside a process.**
- Multiple threads can exist within the same process and share resources such as **memory**, while different processes do not share these resources.
- Like process states, threads also have states:
  - New, Ready, Running, Waiting and Terminated
- Like processes, the OS will switch between threads (even though they belong to a single process) for CPU usage
- Like process creation, thread creation is supported by APIs
- Creating threads is inexpensive (cheaper) compared to processes
  - They do not need new address space, global data, program code or operating system resources
  - Context switching is faster as the only things to save/restore **are program counters, registers and stacks**

Processes and Threads (cont.)

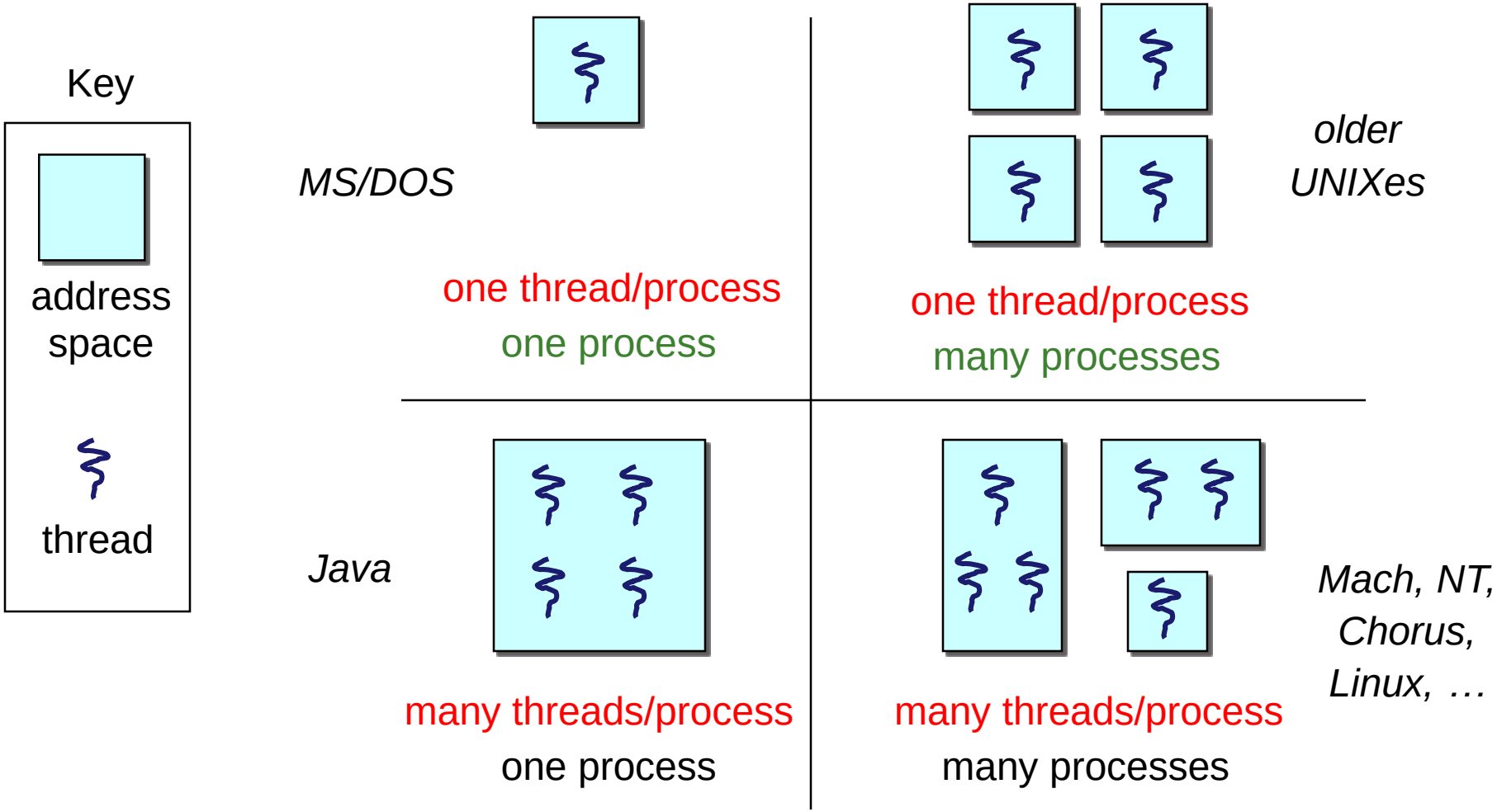
Similarities

- Both share CPU and only one thread/process is active (running) at a time.
- Like processes, threads within a process execute sequentially.
- Like processes, thread can create children.
- Like process, if one thread is blocked, another thread can run.

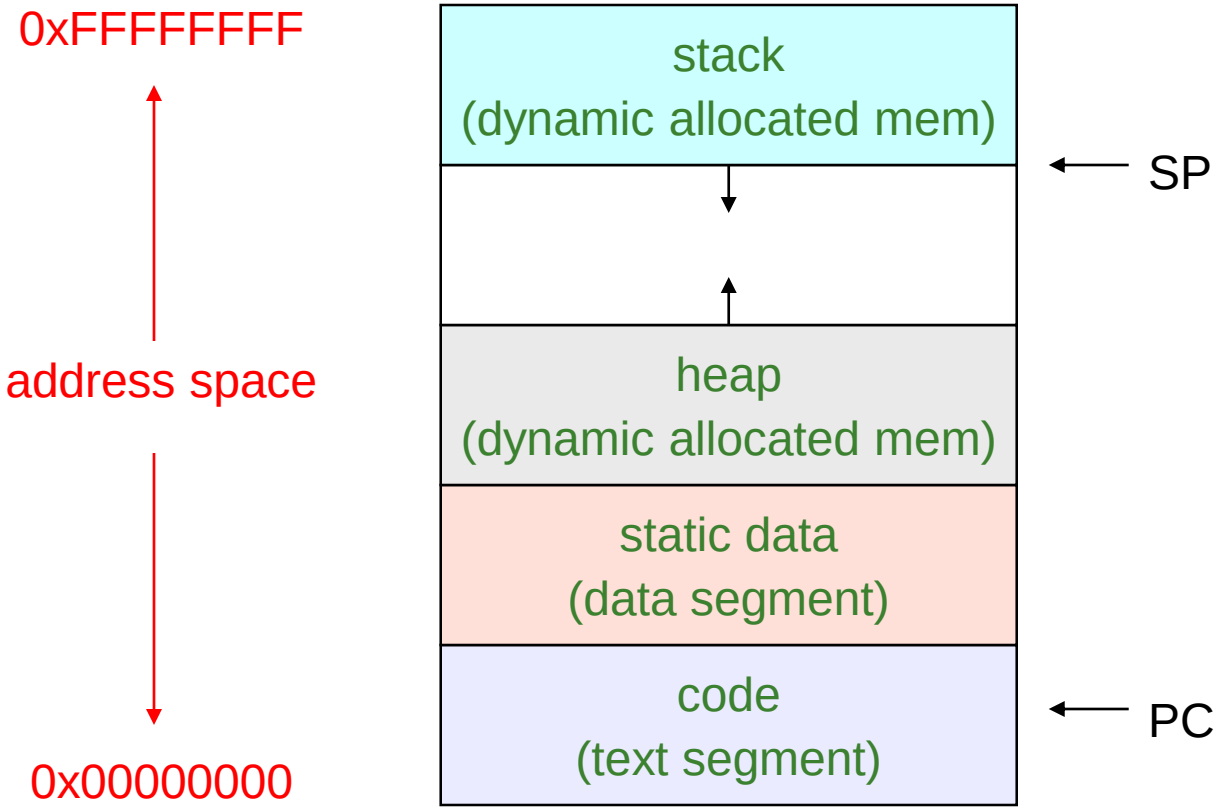
Differences

- Unlike processes, threads are not independent of one another.
- Unlike processes, all threads can access every address in the task.
- Unlike processes, thread are design to assist one other.

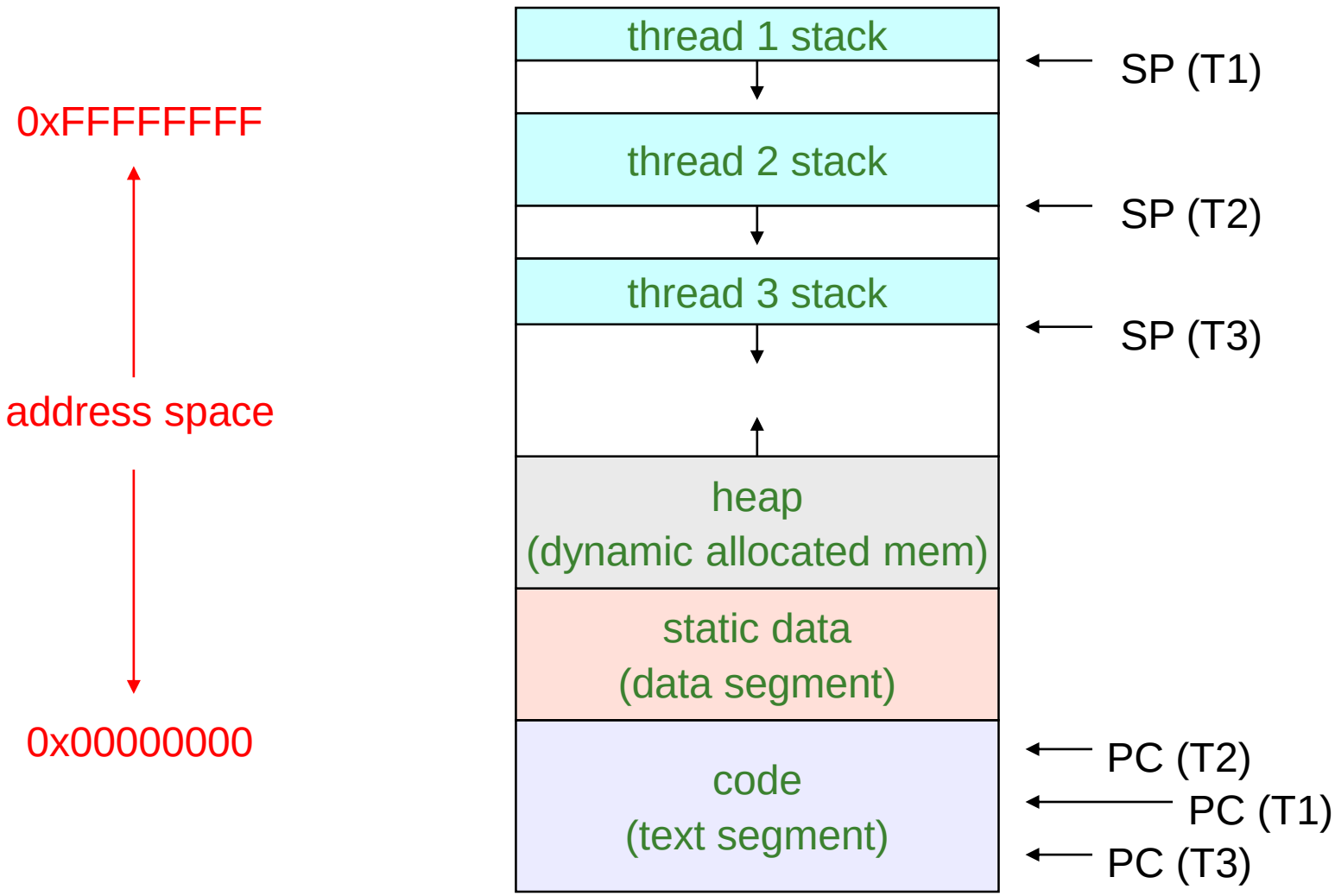
The design space: Process vs Thread



(old) Process address space



(new) Process address space with threads



Why we need threads

- The main disadvantage of single threaded processes is that they don't provide proper utilization of CPU.
- Multithreaded processes can simultaneously perform different functions like accepting requests at the network, doing I/O, doing computation on behalf of a user; each in a separate thread of control.
- Think about a web server, which have thousands of clients sending requests for different parts of a website. If there was only one server side thread running on the server, How much time will be needed to serve the request of each client?
- The effective delay for each response will be very high in single threaded processes

Advantages of multithreaded processes

There are four main advantages of building multithreaded applications:

- **Responsiveness:** For example in case of GUI based user interface, the time between two clicks is expected to be un-noticeable to a user. By providing a separate thread for various buttons and menus the response time is decreased. one thread can give response while other threads are blocked or slowed down doing computations
- **Resource sharing:** Threads are lightweight in nature while processes are heavyweight. It means that, sharing of resources occupied by a process among various threads is possible in multithreaded environment. Threads share common **code, data** and resources of the process to which they belong. This allows multiple tasks to performed within the same **address space**

- **Economy:** creating, managing and context switch of processes is more costly as compared to threads.
- **Scalability:** A single threaded process runs only on single processor regardless of availability of multicore architecture. Now a days, multicore and hyper-threaded architecture allows multiple concurrent threads running in parallel. This kind of dual benefit of parallelism and concurrency provide heavily scalable systems, as the processor cores can be easily added into architecture.

Thread Libraries

- Thread libraries provide programmers an API to create and manage threads
- They can be implemented in either user space or kernel space

User space libraries

- All code and data structure of the library are in user space
- Invoking a function in the library results in a local function call

Kernel space libraries

- Code and data structure of the library is in kernel space
- Invoking an API for the library results in a system call to the kernel

Thread Libraries (cont.)

There are three basic libraries used:

- ✓ **POSIX pthreads**
  - They may be provided as either a user or kernel library, as an extension to the POSIX standard
  - Systems like Solaris, Linux and Mac OS X implement pthreads specifications
- ✓ **WIN32 threads**
  - These are provided as a kernel-level library on Windows systems.
- ✓ **Java threads**
  - Since Java generally runs on a Java Virtual Machine, the implementation of threads is based upon whatever OS and hardware the JVM is running on, i.e. either Pthreads or Win32 threads depending on the system.

Examples of Threads

In a word processor,

- a background thread may check spelling and grammar, while a foreground thread processes user input (keystrokes), while yet a third thread loads images from the hard drive, and a fourth does periodic automatic backups of the file being edited

In a spreadsheet program,

- one thread could display menus and read user input, while another thread executes user commands and updates the spreadsheet

In a web server,

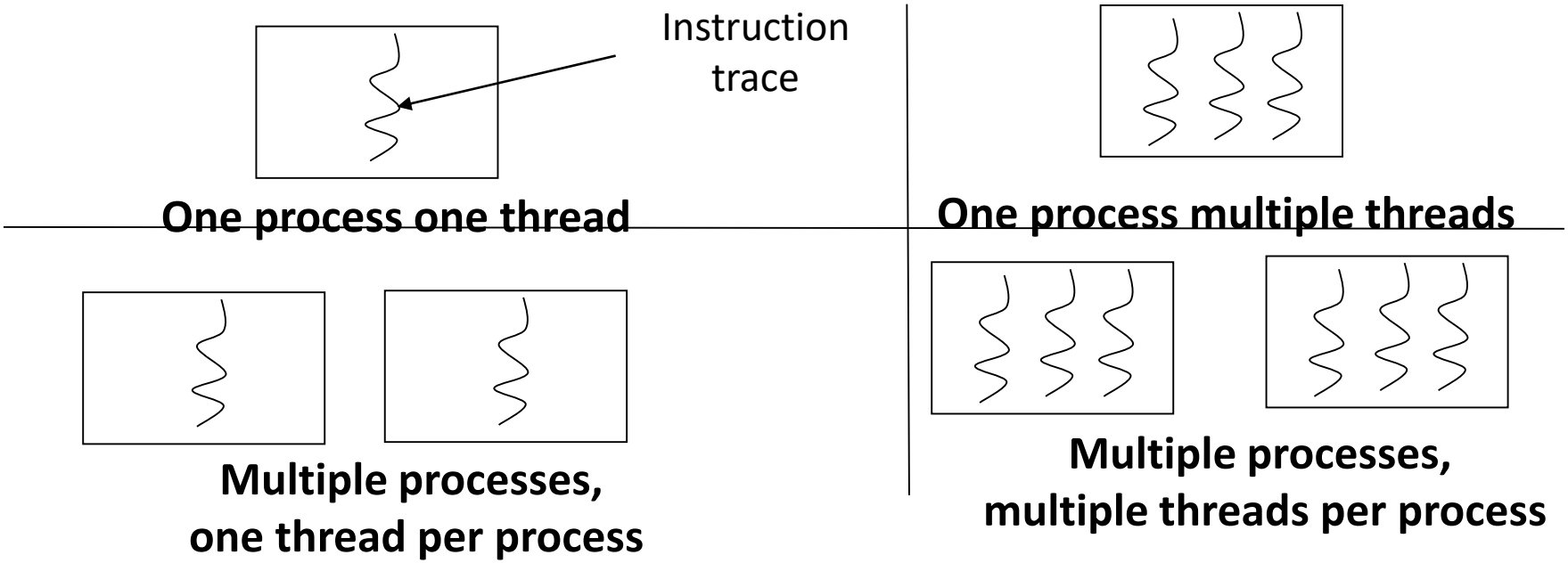
- Multiple threads allow for multiple requests to be satisfied simultaneously, without having to service requests sequentially or to fork off separate processes for every incoming request.

Multithreading

- **Multithreading** refers to the ability on an operating system to support multiple threads of execution within a **single process.**
- A traditional (heavy weight) process has a single thread of control
    - There's one program counter and a set of instructions carried out at a time
- If a process has **multiple thread of control**, it can perform more than one task at a time
    - Each threads have their own program counter, stacks and registers
    - But they share common code, data and some operating system data structures like files

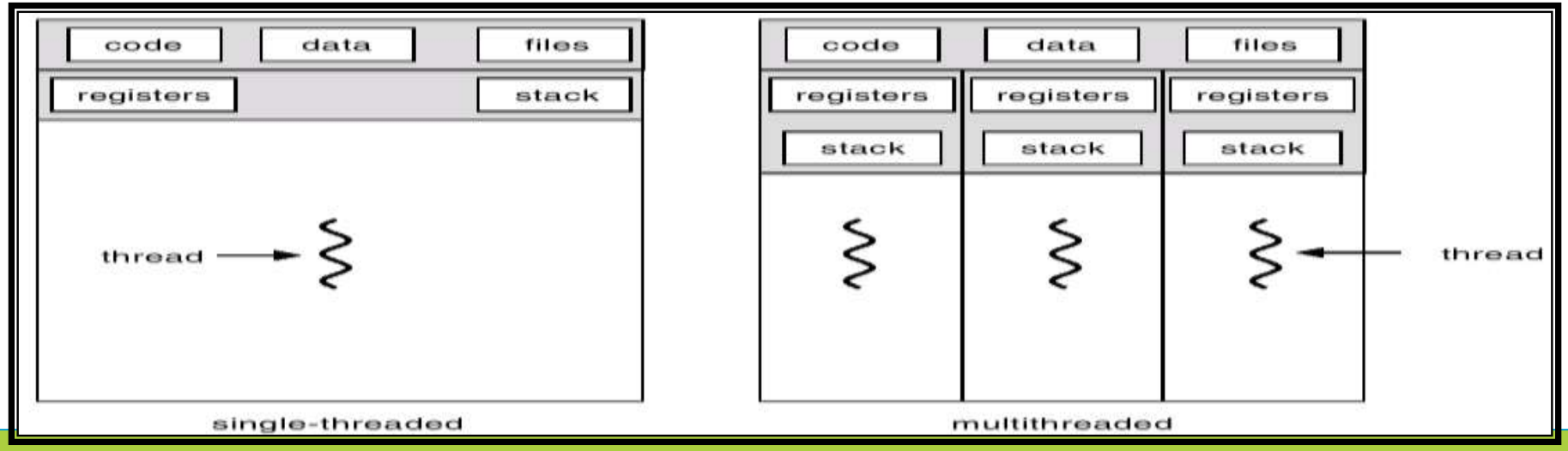
Multi-threading(cont…)

- **Traditionally there is a single thread of execution per process.**
  - **Example: MSDOS** supports a single user process and single thread.
  - **UNIX** supports multiple user processes but only **support one thread per process**.
- **Multithreading**
  - Java run time environment is an example of one process with multiple threads.
  - **Examples** of supporting multiple processes, with each process supporting multiple threads
    - Windows 2000, Solaris, Linux, Mach, and OS/2



Single and Multithreaded Processes

- In a single threaded process model, the representation of a process includes its **PCB, user address space, as well as user and kernel stacks.**
- When a process is running. The contents of these registers are controlled by that process, and the contents of these registers are saved when the process is not running.
- **In a multi threaded environment**
  - There is a single PCB and address space,
  - However, there are separate stacks for each thread as well as separate control blocks for each thread containing register values, priority, and other thread related state information.



Multithreading Models

There are two types of multithreading models in modern operating systems: **user threads** & **kernel threads**

1. **Kernel threads**
   - are supported by the OS kernel itself. All modern OS support kernel threads
   - Need user/kernel mode switch to change threads

2. User threads

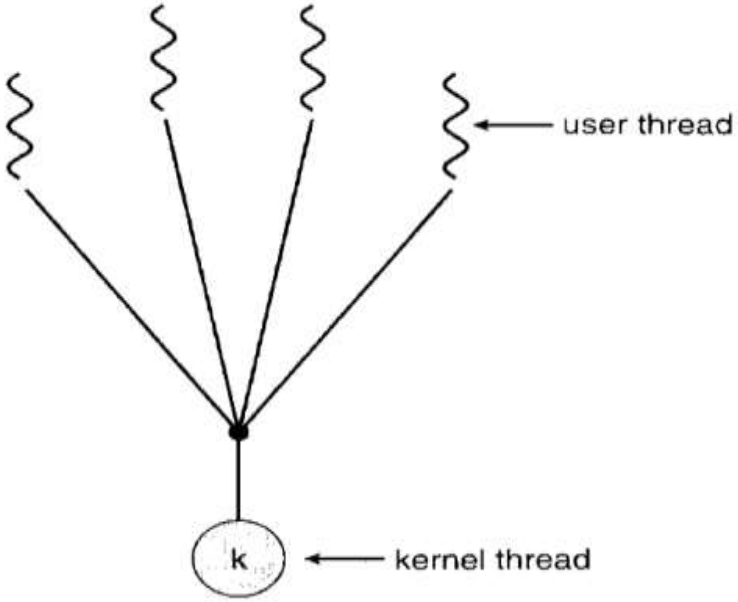
- are threads application programmers put in their programs. They are managed without the kernel support, Does not require O.S. support
- Has problems with blocking system calls
- Cannot support multiprocessing

- There must be a relationship between the kernel threads and the user threads.
- **There are 3 common ways to establish this relationship:**

1. Many-to-One
2. One-to-One
3. Many-to-Many

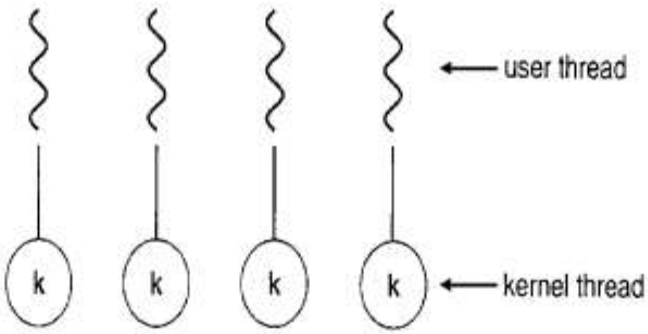
Multithreading Models: Many-to-One

- It maps many user level threads in to one kernel thread
- Thread management is done by thread library in user space
- It's efficient but if a thread makes blocking system call, it blocks
- Only one thread access the kernel at a time, so multiple threads can not run on multiprocessor systems
- Used on systems that do not support kernel threads.



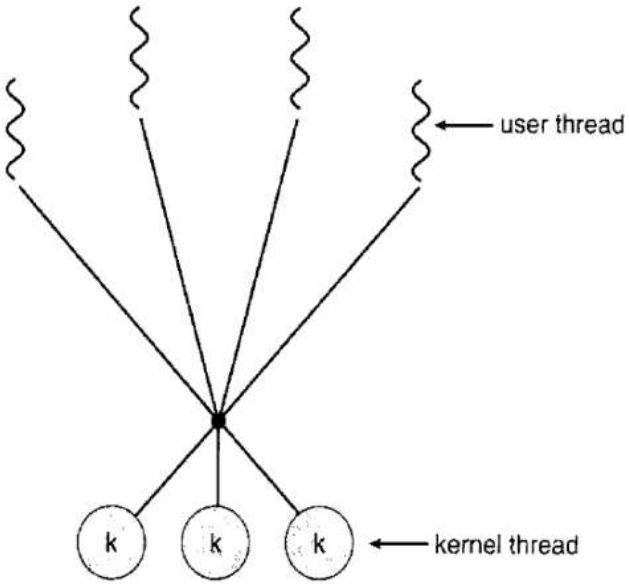
Multithreading models: One-to-One

- Each user-level thread maps to kernel thread.
- A separate kernel thread is created to handle each user-level thread
- It provides more concurrency and solves the problems of blocking system calls
- Managing the one-to-one model involves more **overhead slowed down system**
- **Drawback:** creating user thread requires creating the corresponding kernel thread.
- Most implementations of this thread puts restrictions on the number of threads created



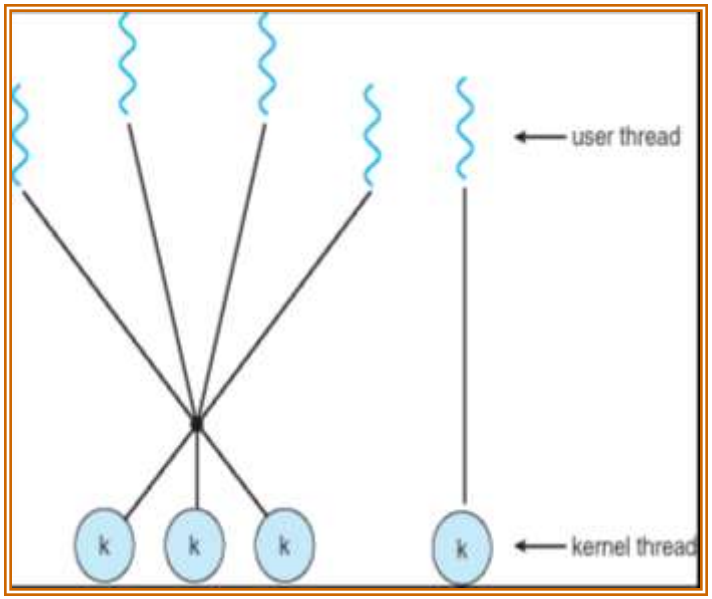
Multithreading Models: Many-to-Many

- allows the mapping of many user threads in to many kernel threads
- Allows the OS to create sufficient number of kernel threads
- It combines the best features of one-to-one and many-to-one model
- Users have no restrictions on the numbers of threads created
- Blocking kernel system calls do not block the entire process.



Multithreading Models: Many-to-Many

- Processes can be split across multiple processors
- Individual processes may be allocated variable numbers of kernel threads, depending on the number of CPUs present and other factors
- One popular variation of the many-to-many model is the two-tier model, which allows either many-to-many or one-to-one operation.



Two-tier model

Threading Issues

- **Some of the issues to be considered for multi-threaded programs**
  - Semantics of fork() and exec() system calls.
  - Thread cancellation.
  - Signal handling
  - Thread pools
  - Thread-specific data
  - Hogging the CPU

Threading Issues in multithreading

- **Semantics of fork() and exec() system calls**
  - In multithreaded program the semantics of the fork and exec systems calls change.
  - If one thread calls fork, **there are two options.**
    - New process can duplicate all the threads or new process is a process with single thread
    - Some systems have chosen two versions of fork
- **Thread cancellation**
  - Task of terminating thread before its completion
    - Example: if multiple threads are searching database, if one gets the result others should be cancelled
  - **Asynchronous cancellation**
    - One thread immediately terminates the target thread
  - **Deferred cancellation**
    - The target thread can paradoxically check if it should terminate

Threading Issue: Thread signaling

- Signals are used in UNIX systems to notify a process that a particular event has occurred
- A **signal handler** is used to process signals
  1. Signal is generated by particular event
  2. Signal is delivered to a process
  3. Signal is handled
- **Options:**
  - Deliver the signal to the thread to which the signal applies
  - Deliver the signal to every thread in the process
  - Deliver the signal to certain threads in the process
  - Assign a specific thread to receive all signals for the process

Threading Issue: Threading Pool

- Creating new threads every time one is needed and then deleting it when it is done can be inefficient, and can also lead to a very large (unlimited) number of threads being created.
- An alternative solution is to create a number of threads when the process first starts, and put those threads into a ***thread pool***.
  - Threads are allocated from the pool as needed, and returned to the pool when no longer needed.
  - When no threads are available in the pool, the process may have to wait until one becomes available.
- The (maximum) number of threads available in a thread pool may be determined by adjustable parameters, possibly dynamically in response to changing system loads.
- Win32 provides thread pools through the "PoolFunction" function.
- Java also provides support for thread pools.

Threading Issue: Thread- specific data

- Most data is shared among threads, and this is one of the major benefits of using threads in the first place.
- However sometimes threads need thread-specific data also in some circumstances
- Such data is called thread-specific data.
- Most major thread libraries (pThreads, Win32, Java) provide support for thread-specific data.

Threading Issue: Scheduler activation

- Both M:M and Two-level models require communication to maintain the appropriate number of kernel threads allocated to the application
- Scheduler activations provide **upcalls** - a communication mechanism from the kernel to the thread library

- This communication allows an application to maintain the correct number of kernel threads
- Hogging: threads struck beyond time allowed to them on CPU

How to keep a user-level thread from hogging the CPU?

- **Strategy 1:** force everyone to cooperate
  - a thread willingly gives up the CPU by calling **`yield()`**
  - **`yield()`** calls into the scheduler, which context switches to another ready thread
  - what happens if a thread never calls **`yield()`**?

- **Strategy 2**: use preemption
  - scheduler requests that a timer interrupt be delivered by the OS periodically
    - usually delivered as a UNIX signal (`man signal`)
    - signals are just like software interrupts, but delivered to user-level by the OS instead of delivered to OS by hardware
  - at each timer interrupt, scheduler gains control and context switches as appropriate